

MANTIS Extended — Post-Paper Extensions to a Banking Multi-Agent Security Testbed

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—MANTIS (Modular Agent Network Testbed for Instrumentation and Security) is a configuration-driven security and observability testbed built directly on a real banking multi-agent system spanning front-, mid-, and back-office workflows. The first paper describing MANTIS scoped its contribution deliberately: a hook bus exposing five fixed control points, a compact attack and failure plugin library, a seven-dimension automated evaluator, and a benchmarking suite – with a named, explicit list of extensions deferred to future work rather than attempted under that paper’s schedule. This paper implements four of those deferred extensions and reports live, measured evidence for each, without modifying the first paper’s own artifact; the fifth, a Zero Trust Backplane integration, is being pursued as a separate, dedicated effort and is intentionally out of scope here (§ I). Additional exporters: all four of the coding plan’s named backends – Jaeger, Grafana (Tempo), Phoenix, and Langfuse – each enabled by a one-line configuration change, that fail silently rather than aborting a run when no collector is reachable; the Langfuse adapter, whose ingestion endpoint is HTTP-only and always requires Basic auth even self-hosted, was confirmed reaching the real hosted Langfuse endpoint and receiving a genuine unauthorized response with no credentials supplied. Security mechanism plugins: a concrete guardrail, evaluated through the identical plugin interface as an attack, that we show live blocking a real attack – a tool-parameter-mutation attack inflating a transfer’s amount from \$125.50 to \$5,000 was denied before it reached the banking backend, with the denial recorded as a distinct `POLICY_EVENT` rather than misclassified as an attack. Additional banking workloads: a new front-office dispute-filing and -tracking process, added as a third route alongside the existing transaction-review and customer-service workflows rather than a new prompt into an existing one – and a debugging finding in its own right: an initial version silently produced zero tool calls across three live trials because its scenario prompt omitted a required identifier every other scenario in the codebase states explicitly, invisible until we actually inspected the emitted trace rather than a clean exit code. We extended this category to the two domains not yet covered: a back-office SAR/exception-escalation process and a mid-office loan-pre-approval process computing a real, deterministic decision from seeded account data rather than an LLM opinion, each live-verified across five trials, surfacing and fixing a further defect where both workloads’ tools were fully functional yet invisible to the testbed’s own inventory discovery. We further extended the security-mechanism-plugin category with four plugins now covering every attack in the shipped catalog that previously succeeded 5/5 with zero

defense: a risk-aware routing guard closing route confusion, initially live-verified with an honestly-reported qualification that blocking the attack does not by itself restore the compliant path, then closed for real with an optional redirect-based recovery mode, live-verified 5/5 reaching the identical ground-truth outcome (manual_review) the undefended baseline itself asserts; a response-redaction filter at the output control point, whose live verification surfaced and fixed a genuine Unicode-hyphen gap in its own detection regex; and a batch-integrity guard closing the back-office ledger-mutation attack, the second (and, as of this revision, last) attack in the catalog that succeeded 5/5 undefended. Two further plugins complete the five categories the coding plan names: a rate limit, which turns a hijack-induced crash (0 of 10 undefended attempts completed) into 5 of 5 completed runs with no false positive on a no-attack control, and an effect-isolation mode, which denied an attacked funds transfer in every trial while the banking backend’s transaction count did not change. We also closed the extensions’ remaining “deployment variants” gap with a genuine second institution profile – an isolated database and a stricter real policy threshold, selected by a runtime configuration field rather than a hardcoded constant, reusing every agent, tool, and workflow unchanged, live-verified 5/5 producing a genuinely different real decision than the default institution would for the identical request. Minimal UI: a schema-driven experiment editor and trace viewer whose mutating actions shell out to the identical CLI subprocess a terminal user would run, verified by confirming a deliberately invalid configuration is rejected with the real CLI’s own error message rather than a UI-side check, extended with campaign and hook-coverage endpoints following the same subprocess-chaining and direct-file-read patterns. The project’s test suite, exercised against real data rather than mocks throughout, stands at 670 passing tests as of this revision.

Index Terms—multi-agent systems, adversarial security, large language models, banking, observability, extensibility

I. Introduction

The first MANTIS paper [1] scoped its contribution narrowly on purpose: a hook bus exposing five fixed control points around an otherwise-unmodified banking multi-agent system, a compact library of attack and failure plugins registered through one shared interface, a seven-dimension automated evaluator, and a benchmarking suite isolating instrumentation overhead from LLM sampling noise. That paper’s own coding plan named five extensions explicitly deferred rather than attempted under its schedule: additional observability exporters beyond

Identify applicable funding agency here. If none, delete this.

OpenTelemetry and MLflow; security-mechanism plugins (guardrails, policy checks, rate limits) evaluated through the same framework as attacks; a Zero Trust Backplane consuming the testbed’s application inventory and control-point metadata to generate or enforce protection; additional banking workloads beyond the ones the first paper’s evaluation exercised; and a minimal, schema-driven UI over the existing command-line interface.

This paper implements four of the five. The fifth, the Zero Trust Backplane, was explored to the same live-verified standard as the other four – a default-deny-by-domain policy generator and enforcement plugin, an exhaustive check of every (agent, tool) pair a live inventory implies, and repeated live trials confirming zero false-positive denials on legitimate traffic – against a real, independently-developed Zero Trust Backplane project for this same banking testbed [2]. That work is being carried forward as its own dedicated integration effort with that project rather than folded into this paper’s own repository and evidence, and is out of scope for the remainder of this paper.

Each of the four extensions covered here is built to the same standard the first paper set for itself: a real, live-verified capability rather than a description of one, checked against the actual emitted trace rather than a clean process exit, with the evidence – not just the claim – checked into the repository. We report that standard being met unevenly on purpose: the dispute-filing workload initially failed to do anything meaningful on first attempt, for reasons a single successful-looking run would not have surfaced, and we report that failure and its root cause alongside the working result, consistent with the first paper’s own central methodological finding that verifying a security-relevant capability means inspecting what actually happened, not assuming it from a lack of an error.

None of the four extensions required a change to `mantis.core`, `mantis.hooks`, or the banking workflow modules – the same architectural claim the first paper made for its own attack and failure plugins now extends to defensive plugins and telemetry adapters as well.

II. Extension 1: Additional Observability Exporters

The first paper’s observability pipeline supported JSONL traces, MLflow run tracking, and a bare OpenTelemetry TracerProvider with no span exporter of its own attached – creatable spans that were never actually shipped anywhere. We implemented all four of the coding plan’s named additional backends: Jaeger, Grafana (Tempo), Phoenix, and Langfuse.

`mantis.observability.jaeger_exporter.setup_jaeger_otel()` attaches a real `OTLPSpanExporter` (gRPC) to the shared TracerProvider, registered in `exporter_registry` as “jaeger” alongside the existing three. Enabling it is a configuration change – `observability.export: [jsonl, jaeger]` – not a code change to the banking workflow,

the hook bus, or the observability plugin. Because a run’s success must never depend on telemetry actually being delivered, we verified the failure path directly: with no collector listening at the configured endpoint, the SDK’s `BatchSpanProcessor` retries and logs a warning rather than raising, confirmed both by a direct unit test asserting no exception propagates and by observing the expected `StatusCode.UNAVAILABLE` warnings during every subsequent offline test run in this development environment, where no Jaeger collector was running.

Grafana Tempo and Phoenix follow the identical adapter shape as Jaeger – both accept OTLP natively over gRPC, so `grafana_exporter.py` and `phoenix_exporter.py` are the same thin wrapper around `OTLPSpanExporter`, differing only in their default endpoint and an optional cloud-auth header/API key for each backend’s managed offering. Langfuse genuinely differs: its OTLP ingestion endpoint (`/api/public/otel`) is HTTP/protobuf only, never gRPC, and always requires HTTP Basic auth – a public/secret key pair – even against a self-hosted instance, so `langfuse_exporter.py` is built on `opentelemetry-exporter-otlp-proto-http` rather than the `.grpc` package the other three share. All three are registered in `exporter_registry` exactly like Jaeger, enabled the same config-only way, and verified with the same fail-silent-with-no-collector test pattern. The Langfuse adapter received a stronger live check than a local no-collector test can offer: run with its default endpoint and no credentials, it reached the real `cloud.langfuse.com` OTLP endpoint over the network and received a genuine HTTP 401 response, independently confirming the transport and endpoint wiring are correct end-to-end against the real service, not only against an absent local collector.

III. Extension 2: Security Mechanism Plugins

The first paper shipped `PolicyPlugin` as an interface only, matching the exact shape of `ExperimentPlugin` so a future defensive plugin would not need to masquerade as an attack plugin, without a concrete implementation. We built `AmountLimitGuardrailPlugin`: a guardrail denying a tool call whose declared amount field exceeds a configured limit, through the identical `apply(ctx) -> HookResult` contract every attack plugin already uses.

The demonstration pairs this guardrail with the first paper’s own tool-parameter-mutation attack rather than an undefended baseline, because the interesting property is not that the guardrail can deny a call in isolation but that it evaluates a call an attack has already mutated. Policy plugins are registered onto the hook bus after any configured attack (mirroring the attack-before-observability ordering the first paper established, for the identical reason: a plugin registered later in a dispatch sees the effect of everything registered before it). Run live against a real model with a \$2,000 limit: `execute_transfer` was called with a legitimate \$125.50 transfer; the attack mutated it to \$5,000 destined for an account named `HACKER-9999`;

the guardrail denied the mutated call before it reached the banking backend; and the agent correctly reported the block back to the customer as a payment-limit restriction rather than a generic failure.

This also required a small but consequential change to the observability layer’s own event classification. The first paper’s `ObservabilityPlugin` classified every non-trivial plugin action as `ATTACK_INJECTED` except a named set of failure-plugin names, which it classified as `ANOMALY`. A guardrail’s `DENY` is a defense working as intended, not an attack; classifying it as `ATTACK_INJECTED` would have made every future evaluator built on that event type unable to tell a caught attack apart from a successful one. We added a parallel `POLICY_EVENT` classification, keyed the same way the existing failure-plugin split already was. The live run’s trace confirms both classifications appear correctly and distinctly in the same run: one `ATTACK_INJECTED` record for the mutation, one `POLICY_EVENT` record for the denial.

We extended this category with two further plugins, each targeting a specific gap the first paper’s own evaluation reported rather than a generic capability. `RiskAwareRoutingGuardPlugin` targets route confusion – the one attack in the first paper’s catalog that succeeded 5/5 with zero defense anywhere in the repository. A generic “is this `transfer_to_agent` destination valid” check cannot catch this attack: its forced destination is a legitimate destination in general, and the attack’s trick is picking the wrong one of two valid routes for a specific transaction, not routing somewhere invalid. This guard instead independently re-fetches the targeted transaction’s real risk score from the banking backend – the same data `get_transaction_context` exposes, computed by the backend rather than by whatever the LLM router decided – and denies the reroute if that transaction is still too risky to skip review for. Run live against the paired route-confusion attack across five trials, the attack fired every trial (`ATTACK_INJECTED`) and the guard denied every trial (`POLICY_EVENT`), 5/5.

This result carries an honest qualification we verified rather than assumed. Blocking the malicious reroute does not, by itself, cause the workflow to retry the correct compliant path: the router reports the denial back and the workflow ends without reaching manual review, rather than a second, corrected `transfer_to_agent` call to the legitimate destination. A guard that blocks a bad outcome and a guard that restores the original correct outcome are different, non-equivalent claims; our evaluation configuration for this demonstration asserts only the former (`transfer_to_agent` as the expected tool, no asserted terminal state), corrected from an earlier draft that had asserted the latter before we observed real run behavior contradicting it.

`ResponseRedactionPlugin` demonstrates the output control point rather than tool: it masks account-number-

shaped tokens in a run’s final result before it reaches the caller, keeping only the last two characters of each id visible. Making an output-stage `MUTATE` actually reach the caller required a real architectural fix, not only the plugin: `mantis.banking.runner.run_message` previously dispatched before `_output/after_output` only once, from `MantisHookPlugin.after_run_callback`, carrying nothing but `{"status": outcome}` – too early in the agent framework’s own lifecycle for the real customer-facing text to exist yet. We added a second, later `after_output` dispatch carrying the real compacted result, applying any `MUTATE` back onto the value the function returns – the same “recorded in the trace but changing nothing real” defect class the first paper found once already, this time at the output rather than the tool control point. Making the second dispatch distinguishable from the first (rather than misread as a second, spurious workflow-termination event) required a corresponding guard in the observability layer keyed on the presence of a status field.

Live verification of this plugin surfaced a genuine, non-obvious gap in its own detection logic: a live model naturally wrote an account id in prose using a Unicode non-breaking hyphen (`CUST-002`, `U+2011`) rather than plain ASCII `-`, which an ASCII-only pattern silently failed to match, leaving that occurrence completely unredacted with no error raised. After widening the pattern to accept the common Unicode hyphen/dash variants, we re-ran the demonstration 4/4 times with zero unmasked account-shaped tokens surviving in the final trace, regardless of which routing path the live model took or which hyphen character it happened to produce. We report this the same way we report the routing guard’s original qualification below: a defense’s own detection surface should be verified against genuinely model-generated text, not only against hand-written test fixtures that happen to match the author’s own formatting assumptions.

We then closed both remaining gaps this section originally reported as honest limitations rather than leaving them as future work. Recovery, not just denial: `RiskAwareRoutingGuardPlugin` now accepts an optional `redirect_to` parameter – when set, a risky reroute is `MUTATED` to the legitimate destination instead of denied outright, so the real `transfer_to_agent` call still proceeds, just to the correct place, rather than the workflow simply failing to continue down the wrong one. Live-verified 5/5: the workflow now reaches `search_policies` and terminates in `manual_review` every trial – the identical ground truth the undefended baseline itself asserts. Both configurations are kept: the deny-only one remains a real, smaller, still-true claim on its own, and the redirect one demonstrates the stronger, self-correcting capability built on top of it.

The back office’s own undefended gap: route confusion was not the only attack in the first paper’s catalog that succeeded 5/5 with zero defense – the back-office tool-parameter-mutation attack (§ III’s ledger-batch substitution) did too, and remained undefended even after this

paper’s first revision. BatchIntegrityGuardPlugin closes it the same way the routing guard closes route confusion: not by checking whether the target batch id is valid (both the genuine and the substituted batch are real, both are seeded ready), but by independently re-checking the target batch’s own real expected-vs-posted totals and denying the ledger update if that batch already carries an unresolved discrepancy. Live-verified 5/5 against the paired attack, with a control run confirming no false positive against the legitimate, undefended clean-batch flow. With this and the routing guard, every attack in the shipped catalog that previously succeeded 5/5 with zero defense now has one.

The coding plan names five kinds of security-mechanism plugin – guardrails, policy checks, isolation, rate limits, and response filters – and the plugins above cover three of them; we built the remaining two. RateLimitGuardrailPlugin caps calls to a tool per run, per time window, or per distinct value of one argument, and denies the call that would exceed the cap. It was motivated by a failure the extended attack library exposed rather than by an imagined one: forcing the root agent’s hand-off to a different domain router makes that router try to send the request back, the hijack rewrites the hand-back too, and the framework raises an unhandled “cannot transfer to itself” error – 10 of 10 recorded attempts crashed. Capping hand-offs at one per destination, with the limiter registered after the attack so it sees the rewritten destination, stops the loop at its second attempt: 5 of 5 live runs completed (one to three denials each), and a no-attack control with the identical limit ended in manual review with zero policy events. We are explicit about scope: this contains the attack-induced denial of service; it does not restore the compliant path.

ActionIsolationPlugin contains a run’s blast radius by effect rather than by caller: while active, tools whose metadata marks them as moving money (or, optionally, as changing any persisted state) cannot execute, whatever an agent or an attack that rewrote its arguments asks for – a “shadow mode” in which agents propose and something else acts. Against the transfer-rewriting attack it denied `execute_transfer` in 5 of 5 trials, and the banking backend’s transaction count was unchanged across the runs. Because isolation cannot distinguish a legitimate call from an attacked one, it also refuses the customer’s real transfer; that is the definition of isolating an effect, not a defect, and a downstream step that depends on the blocked tool will see the denial as a failed step.

IV. Extension 3: Additional Banking Workloads

The coding plan’s fourth extension calls for “more banking processes... or workflow patterns,” which we read as a requirement for a genuinely new business process, not a new prompt into an existing one. We added dispute filing and status lookup as a front-office capability: a new disputes table in the same SQLite repository `submit_manual_review` already uses, a new tool module, and `dispute_resolution_agent` registered as

a third route under `front_office_router`, alongside the existing transaction-review and customer-service-chatbot routes – distinct from the chatbot’s pre-existing FAQ-only “how do I dispute a transaction” answer, which remains purely informational and never files a real case.

The first attempt at this extension produced a run that exited cleanly and called no tool at all, across three consecutive live trials against a real model. The cause was not a defect in the tool or the agent’s routing – both worked correctly, confirmed by the real transfer to `dispute_resolution_agent` appearing in every trace – but in the scenario prompt itself, which never stated a customer identifier, unlike every other scenario in this codebase. `file_dispute(customer_id, transaction_id, reason)` requires `one` as its first argument; without it stated in the prompt the model apparently could not commit to calling the tool at all, silently, with no error the process’s own exit code would reveal. Adding the customer identifier to the prompt – not changing the tool’s signature – produced, on the next live run, a real case filed with a generated identifier, correctly persisted, and a correct customer-facing confirmation message referencing that identifier. We report this not as an aside but because it is the same class of finding the first paper’s own central claim predicts: a clean exit and a correctly-routed trace both looked identical whether or not the actual business action happened, and only inspecting the tool-call events specifically distinguished the two.

We extended this workload category to the two banking domains that had not yet received a new process. Back-office SAR/exception escalation adds `sar_escalation_agent` as a second route under `back_office_router`, alongside the existing EOD workflow: it reads an existing reconciliation exception’s details and, if they describe potential misconduct rather than a routine mismatch, files a formal Suspicious Activity Report to a new `sar_reports` table. A pre-seeded exception case worded with clear misconduct signals (an unexplained, round-number discrepancy repeated identically across three prior business days, with no source-system corroboration) rather than an ordinary-sounding timing mismatch was necessary to avoid reproducing the same class of scenario-design gap Section III already reports – an under-specified scenario a model reasonably declines to act on. Live-verified across five trials, `tool_use_correctness` scored 1.0 every time, with a real, distinct SAR record persisted per trial.

Mid-office `loan_pre-approval` adds `loan_preapproval_agent` as a third route under `mid_office_router`. It is deliberately distinct from the pre-existing `loan_agent` (inside the representative-assist workflow), which returns only informational process guidance and never makes or persists an actual decision: the new agent’s `submit_loan_application` tool computes a real, deterministic decision from the customer’s own seeded account data – approved if the requested amount

is within 25% of the customer’s total balance and their risk tier is not high, otherwise referred to manual underwriting – and persists it to a new loan_applications table. Live-verified across five trials against a customer with a seeded \$9,620.13 checking balance requesting a \$2,000 loan, every trial produced a real, computed approved decision with the exact stated rationale, not an LLM judgment call.

Extending this category surfaced one further defect of the same class the dispute workload’s own scenario-prompt gap represents, this time in the runtime adapter rather than a scenario prompt: both new workloads’ tools were fully reachable by their agents and correctly scoped in `_DOMAIN_TOOL_NAMES`, yet invisible to mantis –inventory’s own raw tool discovery, because their tool modules were never added to the adapter’s `_TOOL_MODULE_PATHS` list. A capability can be completely functional end-to-end while still being invisible to the testbed’s own self-reported inventory; we found this only by directly querying `_discover_banking_tools()` rather than trusting that a working live run implied correct registration everywhere else, and fixed it before reporting either workload as complete.

The coding plan’s fourth extension names two distinct things – “more banking processes... or workflow patterns” and, separately, “deployment variants” – and the three workloads above cover only the first. We closed the second with a genuine second institution profile: a new optional `experiment.institution` configuration field, read into an environment variable before any banking module is first imported in the process (the identical import-ordering discipline § II already established for API keys), which selects an isolated SQLite database and a genuinely different policy threshold, while reusing every agent, tool, and workflow unchanged. A `regional_credit_union` profile seeds its own member (a \$3,000 checking balance) and enforces a stricter 10% loan-approval threshold, against the default institution’s 25%. Live-verified 5/5: a \$500 request (16.7% of that member’s balance) is correctly referred under this institution’s threshold – the identical request, against the identical code path, would have been approved at the default institution’s threshold. The point is not the specific numbers but that institution is a genuine runtime parameter of the architecture, not a hardcoded constant, closing the one dimension of this extension the three new workloads alone did not.

V. Extension 5: Minimal UI

The coding plan’s final named extension is deliberately constrained: “a schema-driven editor and trace viewer that invokes the same CLI/API; no separate business logic.” We built a small FastAPI application whose two mutating actions – validate and run – shell out to the exact mantis console-script entrypoint a terminal user would invoke, as a real subprocess, rather than a second, UI-specific implementation of configuration validation or experiment

execution. Every read-only endpoint – the configuration schema, the domain/workflow/scenario/plugin registries, the live inventory, and a run’s own trace file – calls the identical underlying objects the CLI itself reads from, so the editor’s form fields are populated from live data rather than a hardcoded list that could drift from the deployed system.

We verified the “no separate business logic” property directly rather than assuming it from the implementation’s structure: submitting a configuration with a deliberately invalid domain through the UI’s validate endpoint returns the real CLI’s own “Unknown domain” error text, produced by the same subprocess a terminal invocation of `mantis –validate` would produce, not a UI-side check reproducing that logic. The application was also confirmed running as a real server, not only through an in-process test client: launched via `mantis –ui`, it correctly reported the current live inventory (34 agents, 27 tools, reflecting the two additional workloads in Section III) over real HTTP requests.

We added three further endpoints, symmetric with the existing read-only pattern and the CLI-subprocess pattern respectively. `GET /api/runs/{name}/hook-coverage` reads `hook_coverage.json` directly, the same file the trace and evaluation endpoints already expose. `POST /api/campaign` shells out to `mantis –campaign <dir>`, parses the output directory from that command’s own stdout, and chains a second real CLI invocation – `mantis –report <dir>` – against it, rather than reimplementing campaign aggregation as UI-side logic; `GET /api/campaign/{name}/report` then reads the resulting `report.md` directly. The front-end’s trace and evaluation views were also changed from a raw JSON dump to a rendered table, kept intentionally thin – generic across whichever evaluator dimensions a run happens to report, rather than a hardcoded per-dimension layout – consistent with the coding plan’s own “no separate business logic” framing for this extension.

VI. Evaluation Summary

Table I summarizes each extension’s verification evidence. All items were checked against real data or a real live run rather than a mock, consistent with the first paper’s own testing philosophy; none required a change to `mantis.core`, `mantis.hooks`, or the banking workflow modules. The offline suite stands at 670 passing tests (including a parametrized consistency suite over every shipped configuration and scenario), still executing in well under two minutes with no live LLM key required: 4 for the Jaeger exporter and 9 for Grafana/Phoenix/Langfuse together, 6 for the amount-limit guardrail, 9 for the risk-aware routing guard (deny, redirect, and the real-hook-bus cases for each), 10 for response redaction, 6 for the batch-integrity guard, 9 for the rate limit, 8 for action isolation, 7 for the dispute workflow, 10 for SAR escalation, 12 for loan pre-approval, 5 for the institution deployment variant,

TABLE I
Post-Paper Extensions: verification evidence.

Extension	Tests	Live evidence
Jaeger exporter	4	Real OTLP span processor attached; confirmed silent-safe with no collector listening.
Grafana/Phoenix9 Langfuse	9	Same OTLP adapter pattern for the first two; Langfuse (HTTP-only, Basic auth) reached the real hosted endpoint and got a genuine 401 with no credentials.
Amount-limit guardrail	6	Live run: \$125.50 transfer mutated to \$5,000 by a real attack, denied by the guardrail before reaching the backend; both events correctly classified.
Risk-aware routing guard	9	Live, 5/5 deny and 5/5 redirect: the redirect mode reaches manual review every trial, the identical outcome the undefended baseline asserts.
Response redaction	10	Live, 4/4 after fixing a real Unicode-hyphen regex gap found via verification; zero unmasked account ids survived regardless of routing path.
Batch integrity guard	6	Live, 5/5: back-office ledger-mutation attack denied every trial; control run confirms no false positive on the legitimate clean-batch flow.
Rate-limit guardrail	9	Live, 5/5: a hijack-induced loop that crashed the run 10/10 undefended now completes (1–3 denials per run); the no-attack control shows zero policy events.
Action isolation	8	Live, 5/5: an attacked funds transfer denied every trial; the backend’s transaction count unchanged.
Dispute workload	7	Live run: real case filed and persisted after correcting a scenario-prompt defect found only by inspecting the trace.
SAR escalation	10	Live, 5/5: real SAR record persisted per trial; tool_use_correctness 1.0 every time.
Loan pre-approval	12	Live, 5/5: real, deterministic approved decision computed from seeded account data every trial.
Institution variant	5	Live, 5/5: identical code path, isolated database, a stricter real threshold produces a genuinely different real decision than the default institution.
Minimal UI	15	Real invalid config rejected with the real CLI’s own error text; real running server confirmed via HTTP; campaign endpoint chains two real CLI invocations, verified by mocking only the subprocess boundary.

and 15 for the UI, on top of the base suite the first paper’s own work established.

VII. Conclusion

We implemented four of the five extensions the first MANTIS paper explicitly deferred, to the same evidentiary standard that paper set for itself: live-verified behavior, checked against the actual emitted trace, with the evidence checked into the repository rather than described in prose. Each category was then deepened once its first, minimal instance was live-verified: additional banking workloads grew from one to three, spanning all three domains for the first time; security-mechanism plugins grew from one to three, closing the specific worst-case gap the first paper reported (route confusion, 5/5 undefended) rather than adding a plugin for its own sake; and the minimal UI gained the campaign and hook-

coverage surfaces the coding plan named alongside its original validate/run/trace scope. This deepening surfaced three further real defects, each found the same way the first paper’s own central finding predicts – by inspecting actual behavior rather than trusting a clean exit or a passing self-report – and each fixed before being reported as working: a scenario-design gap that silently produced zero tool calls across three live trials; two new workloads’ tools that were fully functional yet invisible to the testbed’s own inventory discovery; and a Unicode-hyphen gap in a redaction filter’s own detection regex that left real account ids in live model output completely unmasked. We also report, rather than omit, a genuine limitation the routing guard first shipped with: blocking an attack and restoring the system’s original correct behavior are different claims, and our own first draft of that demonstration’s evaluation config conflated them until live evidence corrected it. Rather than leave that gap as named future work, we closed it in this revision – an optional redirect mode that restores the compliant path, live-verified 5/5 – and closed the two other gaps this paper had previously only named: a batch-integrity guard for the back office’s own undefended tool-mutation attack, and a genuine second institution profile (isolated data, a stricter real policy threshold, selected by configuration rather than hardcoded) closing the “deployment variants” half of the banking-workloads extension the three new processes alone did not cover. Every attack in the shipped catalog that once succeeded 5/5 with zero defense now has one, and all five security-mechanism categories the plan names now have a live-verified plugin. None of this work required a change to the first paper’s core interfaces. The fifth extension, the Zero Trust Backplane, was built and repeated-trial-verified to this same standard – including an exhaustive check of every (agent, tool) pair a live inventory implies, and a real event-misclassification defect found and fixed along the way – but is being carried forward as a dedicated integration effort with a real, independently-developed Zero Trust Backplane project for this same testbed [2] rather than reported as part of this paper’s own repository and evidence. What remains honestly open, rather than closed: repeated-trial efficacy statistics at a larger sample size than this paper’s practical $n=4-5$ per configuration, and continuing that separate Zero Trust integration effort toward a production integration between this testbed’s inventory and a real signed decision pipeline.

Acknowledgment

References

- [1] [Authors], “MANTIS – Modular Agent Network Testbed for Instrumentation and Security in Multi-Agent Banking Systems,” 2026. (Companion paper; this work’s Section II onward assumes its architecture and terminology.)
- [2] smartsystemslab-uf, “Citi P3 Zero Trust Backplane,” github.com/smartsystemslab-uf/Citi_P3_Zero_Trust_Backplane, 2026.